

UNIVERSITÉ DE NANTES

INTERNSHIP REPORT

Bayesian Network Learning

Author:

Mina RAFLA

Supervisor:

Pr. Philippe LERAY

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Science*

in the

LS2N
Polytech Nantes

May 17, 2026

Acknowledgements

I would really like to express my great appreciation to M. Philippe LERAY for his supervision and for his patience that were always motivating me, especially during the situation of the pandemic of Covid-19. I was really lucky and will always be grateful for having a supervisor that constantly encourages, supports, gives me great confidence and diffuses a great spirit of cooperation among his interns and students.

I would also like to thank M. Hoel LE CAPITAINÉ for always being available to respond to any enquiry and for his great support during all the internship period and during the Master program which helped me to overcome different situations.

Finally, I should also mention my colleagues in the Laboratory of Nantes, Evan Duffrais, Houada Abouchamala, Philipp Behrendt and all the other interns and students for the discussions and exchanges that were very fruitful.

Contents

Acknowledgements	i
1 Laboratory, research Team and Internship's Subject	1
1.1 Laboratory and the research team	1
1.2 The internship's subject	1
2 Concepts Review	3
2.1 Introduction	3
2.2 Graphs, Networks and Bayesian Networks	3
2.2.1 DAG	3
2.2.2 Markov Equivalence[11]	3
2.2.3 Bayesian Networks	3
2.2.4 Chickering's algorithm to convert a DAG to a CPDAG	4
2.3 Evaluation metrics	5
2.3.1 SHD	5
2.3.2 TP, TN, FP, FN	5
2.3.3 Bayesian Network Random Generation using PMMixed	6
2.4 Conclusion	6
3 Parameters Learning From Data	7
3.1 Introduction	7
3.2 Parameter Learning from Complete Data	7
3.3 Parameters Learning from Incomplete Data	8
3.4 Incremental Learning	8
3.5 Contribution: Experiments and Results	9
3.5.1 Introduction	9
3.5.2 Usecases	9
EyeColorGene Usecase example	9
NoisyMax Usecase example	10
Dummy Usecase example	11
3.5.3 Parameters Incremental Learning with complete data	11
3.5.4 Parameters Incremental Learning with incomplete data	12
3.5.5 Parameters Incremental Learning Simulation for "Non-Stationary"-like system	13
3.6 Conclusion	15
4 Non-Incremental Structure Learning	17
4.1 Introduction	17
4.1.1 Score based methods	17
4.1.2 Constraint Based methods	18

4.1.3	Hybrid methods	18
	MMPC and MMPCbar	18
4.2	Contribution	20
4.3	Experiments and Results	22
4.3.1	Original Structure of Asia Usecase	22
	DAG	22
	CPDAG	22
4.3.2	Asia Structure Learning using MWST algorithm	23
	DAG	23
4.3.3	Learnt Structure of Asia Usecase using GS algorithm	24
	DAG	24
	CPDAG	24
4.3.4	Asia Structure Learning using Greedy Search algorithm + Random Init	25
	The DAG of the Learnt Structure	25
	CPDAG of the learnt structure	26
	The Init Random Network Structure by PMMixed	27
4.3.5	Asia Structure Learning using Greedy Search algorithm + Smart Init by MWST algorithm	28
	The Learnt Structure DAG	28
	The CPDAG of the Learnt Structure	28
	The Init Structure generated by MWST	29
4.4	Conclusion	30
5	Incremental Structure Learning	31
5.1	Roure's approach (TOCO and RSS)[17]	31
5.1.1	TOCO Heuristic	31
5.1.2	RSS Heuristic	32
5.2	Incremental MMPC	32
5.3	Calculating Sufficient Statistics[17]	33
5.4	Contribution	36
5.4.1	Sufficient Statistics	36
5.4.2	IMMPC and IMMPCbar	38
	Example:	38
5.5	Conclusion	41
6	Thesis Conclusion	43
	Bibliography	45

Chapter 1

Laboratory, research Team and Internship's Subject

In this chapter I would like to present the laboratory where happens the internship and the DUKe research team and their domain of work as well as my role during the internship at the team

1.1 Laboratory and the research team

The master internship is in the DUKe (Data User Knowledge) team, one of the teams of the laboratory of Nantes (Laboratoire de Science Numériques de Nantes), also known as LS2N.

The Laboratory of Nantes involves 23 teams in different research domains. The main research axes of the laboratory are:

- System Design
- Robotics
- Signal and Image and Language Processing
- Data Science and Decision
- Software and Distributed Systems

The DUKe team aims at proposing querying, mining and learning techniques on different types of data. Also, their research works care about interacting with experts in different domains.

The team is interested in Bayesian networks, Machine Learning, Process Mining, Deep learning, Visualization, Semantic and Knowledge Representation, Databases and many other related fields.

1.2 The internship's subject

The DUKe team, lead by M. LERAY Philippe (my supervisor), developed a library in C++ called PILGRIM. This library deals with the different aspects

of Bayesian Networks and Probabilistic Relational Models.

"PILGRIM General" that deals with Bayesian Networks have several functionalities of learning Bayesian networks Structure and Parameters from data and from experts.

Each of these functionalities currently works separately without being connected nor tested with the other ones.

Also, these functionalities doesn't have the concept of incrementality. This means, that we cannot tune the parameters nor the structure in an incremental way in the case of the availability of new data or information.

My role during the first part of the internship is to :

1. Ensure the correct working of these functionalities by debugging and testing them
2. Add the incrementality to the Parameters learning and the Structure Learning
3. Connect these functionalities to introduce a level of "continuity" between them so the user:
 - (a) A Bayesian network can be constructed manually by the help of experts.
 - (b) When some of the data is available, I can keep the structure done by the experts and learn the Parameters from the data
 - (c) When some extra data is available, I can learn the parameters incrementally
 - (d) When I have a lot of data, I can learn the structure directly
 - (e) Also, after the availability of other data I can learn it incrementally

Chapter 2

Concepts Review

2.1 Introduction

In this chapter, we are going to make a brief review on some of the probability notions, Networks and graphs, and finally on the probabilistic and graphical models.

2.2 Graphs, Networks and Bayesian Networks

A graph is a pair $G = (V, E)$ where V is a set of vertices and E is a set of Edges. A pair (u, v) is a directed edge that goes from vertex u to vertex v . Here, vertex u is the parent of v and v is a child of u .

2.2.1 DAG

A DAG is a directed acyclic graph where the edges are oriented and does not contain any cycles. The Skeleton of a DAG is a graph where all the edges are not oriented.

2.2.2 Markov Equivalence[11]

Two DAGs are Markov equivalence if and only if they contain the same conditional independencies.

In other words, suppose two Bayesian Networks BN1 and BN2. BN1 and BN2 are markov equivalent if they have the same skeleton, same V-Structures and same inferred edges. All the equivalent graphs can be described by a CPDAG (Completed Partially Directed Acyclic Graph) that have the skeleton as all the graphs (Edges that are not part of a V-Structure and their inverses are represented by undirected edges). For example the 3 DAGs in fig. 2.1 are markov equivalent.

2.2.3 Bayesian Networks

In a Bayesian network, each node represents a variable and each variable is represented only by one node. The directed edges represent probabilistic relation between the variables. Each Bayesian Network is composed of:

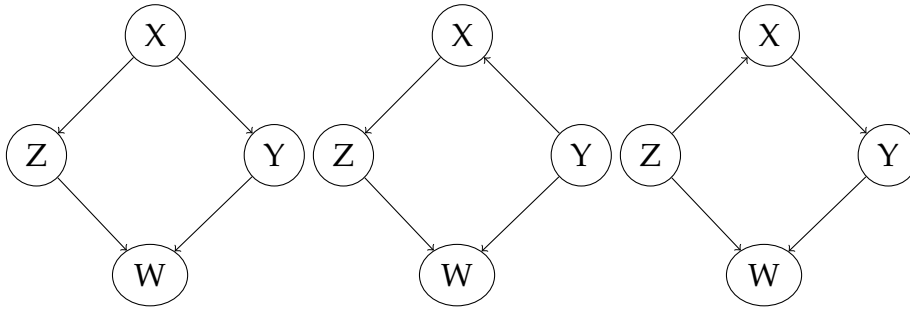


FIGURE 2.1: Three markov equivalent networks

1. Graphical Component: a DAG which encodes a set of dependences and independences.
2. Numerical Component: Set of conditional probability distributions relative to each variable.

2.2.4 Chickering's algorithm to convert a DAG to a CPDAG

To Convert a DAG to a CPDAG, an algorithm by D. Chickering [3] - a scientist in Microsoft Research - was used to label the edges of the graph and do the conversion.

```

Order the edges in the graph;
Label all edges as "UNKNOWN";
while there are Edges Labeled "UNKNOWN" in the graph do
    Let  $x \rightarrow y$  be the lowest ordered edge with "UNKNOWN" label;
    for every edge  $w \rightarrow x$  labeled "COMPELLED" do
        if  $w$  is not a parent of  $y$  then
            Label  $x \rightarrow y$  and every edge incident into  $y$  as
            "COMPELLED";
            Go to the condition of the first while loop;
        end
    else
        Label  $x \rightarrow y$  as "COMPELLED";
    end
end
if exists an edge  $z \rightarrow y$  and  $z \neq x$  and  $z$  not parent of  $x$  then
    Label  $x \rightarrow y$  and all "UNKNOWN" edges incident into  $y$  with
    "COMPELLED"
end
else
    Label  $x \rightarrow y$  and all UNKNOWN
end
    edges incident into  $y$  with "REVERSIBLE"
end

```

Algorithm 1: Chickering's Algorithm

2.3 Evaluation metrics

To compare between 2 DAGs and calculate how close they are to each other, several evaluation metrics can be used.

2.3.1 SHD

To calculate SHD (Structural Hamming Distance) between the original CPDAG and the learnt CPDAG, we checked several conditions that let us know whether to increment the SHD value or not.

Begin with $SHD = 0$

Loop on All edges in OrigGraph and LearntGraph:

IF

1. A Directed Edge in OrigGraph and in different direction or undirected or absent in the LearntGraph

OR

2. An undirected edge in OrigGraph and directed or absent in LearntGraph

OR

3. Edge (Directed or Undirected) in Learnt Graph but absent in OrigGraph

THEN: $SHD \leftarrow SHD + 1$

2.3.2 TP, TN, FP, FN

Calculating TP, TN, FP, FN can be indicators of the similarity between 2 Skeletons.

1. TP (True Positive) The number of the edges in the Original skeleton that we find in the Learned skeleton
2. FP (False Positive) The number of the edges we find in the learned skeleton and that are not in the Original one.
3. TN (True Negative) The number of the edges not found in both skeletons
4. FN (False Negative) The number of the edges in the Original skeletons and are not in the learned one

2.3.3 Bayesian Network Random Generation using PMMixed

To generate a random bayesian network, PMMixed algorithm [9] can be used to generate random graphs with simultaneous constraints like the "induced width", "maximal node degree" and number of edges.

2.4 Conclusion

In this chapter, I introduced some of the basic concepts of probability as well as the BN Comparison metrics (SHD, TP, TN, FP, FN) as well as the chickering algorithm and the PMMixed algorithm. All these algorithms I implemented and integrated in PILGRIM and will be used later as explained in the Parameter and Structure Learning for the Bayesian networks.

Chapter 3

Parameters Learning From Data

3.1 Introduction

One of the goals of this internship is to study and use and implement several aspects of the learning algorithms. We wanted to be able to build a BN in different situations and given different inputs (few data, experts, lots of data, etc.).

So now in PILGRIM:

1. We can build a full bayesian network that describes a problem with the help of experts.
2. Given the Structure of the BN and the Data we can learn the parameters (the probability distribution tables).

The data can be:

- Complete Data
 - Incomplete Data that contains missing values or data without latent variables.
3. Given only the Data we can learn the Structure of the BN and also the data can be:
 - Complete Data
 - Incomplete Data that contains missing values or data without latent variables.

Also, we can apply the concept of Incremental Learning to improve the parameters or structure learnt.

3.2 Parameter Learning from Complete Data

Parameter Learning in a Bayesian Network is the task of learning the parameters and the probability distribution tables for the different variables from the data.

From a complete dataset, we can get the number of occurrences of each value for each variable while fixing the value of another variable. The result of this procedure will be similar to the table [3.1](#).

$X \backslash Y$	a	b
a	15	20
b	30	35

TABLE 3.1: Simple Contingency Table

Given this information, we can calculate the maximum likelihood estimate $\Pr(X = a|Y = a)$ as follows:

$$\Pr(X = a|Y = a) = \frac{\Pr(X = a, Y = a)}{\Pr(Y = a)} =$$

$$\frac{\frac{n(X=a, Y=a)}{N}}{\frac{n(Y=b)}{N}} =$$

$$\frac{n(X = a, Y = a)}{n(Y = a)}$$

From the table 3.1, we can see that $n(Y = a) = 45$ and $n(X = a, Y = a) = 15$.

So estimating the conditional probability will be as follows:

$$\Pr(X = a|Y = a) = \frac{n(X = a, Y = a)}{n(Y = a)} = \frac{15}{45} = 0.333$$

3.3 Parameters Learning from Incomplete Data

The procedure above cannot be applied in case of missing values, because we will not be able to build accurate distribution tables. Here comes the Expectation-Maximization Algorithm (EM)[7] that can be used to calculate maximum likelihood (ML) and maximum a Posterior (MAP).

3.4 Incremental Learning

Incremental Learning[2] takes into consideration passed learnt parameters of the model (the Bayesian network). This is used to enhance the model and tune it as soon as new data is available.

To apply incremental learning we use a forgetting factor α that the user can choose. This parameter decides how much data from the old data to take into consideration for the learning process.

$$\theta_{\text{updated}} = \theta_{\text{old}} * \alpha + \theta_{\text{new}} * (1 - \alpha)$$

Of course if $\alpha = 0$ that means that we ignore any old data and the learning process turns to be a normal learning process from scratch

3.5 Contribution: Experiments and Results

3.5.1 Introduction

The internship's main focus was the learning process for Bayesian networks. The experiments were done by using a C++ library called PILGRIM, which is a library developed by the DUKe Team for Bayesian networks.

3.5.2 Usecases

Creating new usecases and modifying and testing existing ones were the first tasks to do in order to:

1. Help new users get easily familiar to PILGRIM.
2. Use these usecases in our later experiments as we will show here in the report

In this section I would like to present some of the usecases that I implemented or modified.

Each usecase was composed of:

1. Create the Bayesian network for the usecase as an XML file.
2. Create a dataset from the corresponding XML file.
3. Query the Bayesian network
4. Learning the parameters (incrementally and non incrementally) of the BN given the structure and the data.

EyeColorGene Usecase example

The first usecase created was a small case for a Bayesian Network that predicts the eye color of the children.

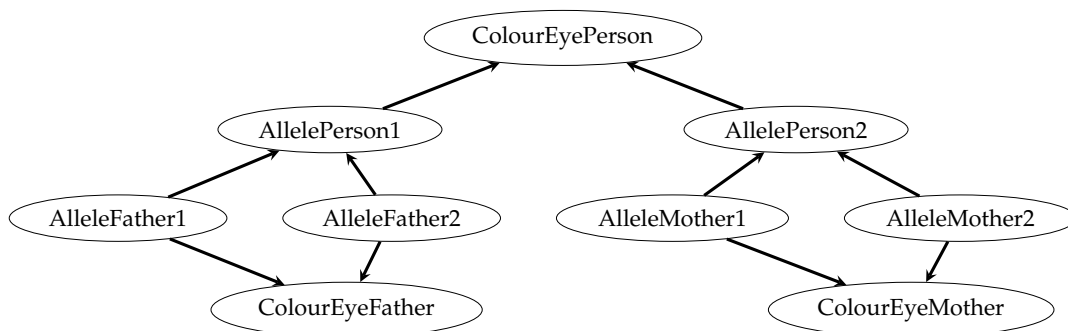


FIGURE 3.1: Eye Color Gene Bayesian Network

NoisyMax Usecase example

The Leaky ICI Model [8] (noisyMax example represented in 3.2) represents an example of a real world problem where not all the variables that influence the output are explicitly included in the model but there are some latent and hidden variables and well as a LEAK variable represented by L variable.

Assume that the variables : A, B, C are symptoms, Y is the output which says whether there is a disease or not. In this case, L is a Leak variable that can determine that there is disease even if all the other symptoms do not exist.

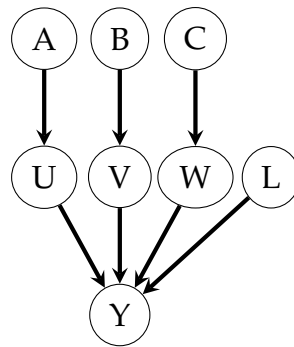


FIGURE 3.2: NoisyMax Bayesian Network containing Latent Variables

In my example here, each of the symptoms variables (A, B, C) here can have one of the 3 values: "ok", "deg", "ko". The probability Tables of the "non-hidden" variables are as follows:

	ok	deg	ko
$\Pr(A)$	0.5	0.4	0.1

	ok	deg	ko
$\Pr(B)$	0.2	0.6	0.2

	ok	deg	ko
$\Pr(C)$	0.7	0.2	0.1

$\Pr(Y)$	Max of W, V, L
----------	----------------

To test that the Bayesian network non-incremental learning works good with incomplete data:

1. Created manually a Noisy Max example with latent variables
2. Generated data from this model in a CSV form
3. Cut the data in order to remove the columns of the hidden variables (U, V, W, L)

4. Begin the Parameter learning process with this new "incomplete" data.

After beginning the learning process, the results were quite satisfactory, the model was learnt and the probability distribution was as follows:

	ok	deg	ko
$\Pr(A)$	0.490	0.406	0.102

	ok	deg	ko
$\Pr(B)$	0.198	0.589	0.212

	ok	deg	ko
$\Pr(C)$	0.708	0.176	0.114

So, as we can see the probabilities converges towards the real ones.

Dummy Usecase example

This was a just simple Bayesian network used to test the incremental learning functionality and a non-stationary system-like example.

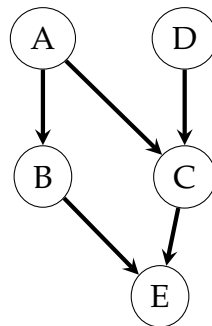


FIGURE 3.3: Simple Dummy usecase used later for testing the incremental learning functionality

3.5.3 Parameters Incremental Learning with complete data

The PILGRIM library already implements the learning algorithms to learn the parameters of a Bayesian Network, given the structure and the Data.

One of the completed tasks was to implement an incremental learning functionality and to test it. The incremental learning functionality now works with a constant forgetting factor specified by the user.

We also tested this functionality with a forgetting factor that auto-increments based on the number of datasets used for the training process. We used the existing learning functions in PILGRIM and modified it in order to make it also work for an already learnt Bayesian network that contains probability tables and their parameters.

A	
T	F
0.3	0.7

TABLE 3.2: Original Probability table of variable A in the Dummy Usecase

We tested this functionality with the simple Dummy usecase in fig. 3.3. The original probability table of the variable A was as shown in table 3.2

Given the structure and the data, we splitted the data to 10 parts, specified a forgetting factor of 0.7 and observed how the values learnt of the probability of the variable $A=\text{True}$ evolves, as shown in fig. 3.4. (Each iteration means learning a new part of the 10 parts of the data)

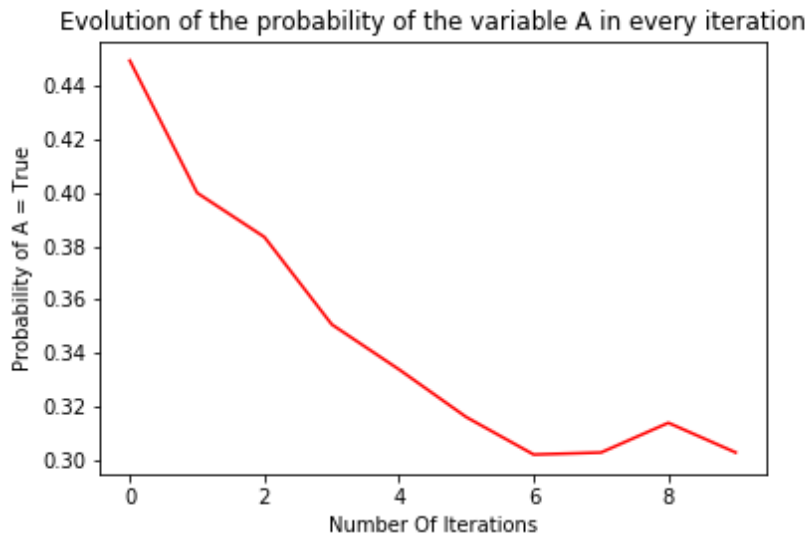


FIGURE 3.4: The evolution of the learnt probability of variable $A = T$

As we can see in fig. 3.4, $\Pr(A)=\text{True}$ was 0.5 approximately during the first iteration and converged to 0.30224 at the last iteration (converged to the original probability value found in Table 3.2).

3.5.4 Parameters Incremental Learning with incomplete data

As shown in section 3.5.2 for the NoisyMax usecase example, we performed the incremental learning process in incomplete data, and also we found that the data converges to the right probabilities as shown in fig. 3.5:

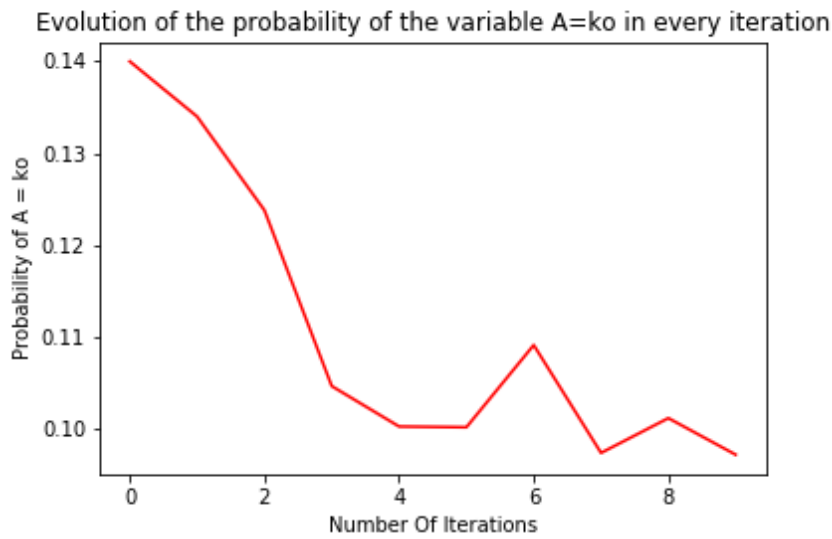


FIGURE 3.5: The evolution of the learnt probability of variable $A = ko$

3.5.5 Parameters Incremental Learning Simulation for "Non-Stationary"-like system

Non-Stationary Systems are unpredictable systems where the parameters are always changing and that cannot be modeled using non-incremental approaches. To simulate a system like this, we created 3 different Bayesian networks, same structure but different probability tables and different datasets that corresponds each to one the BN.

We started learning the parameters given the generated datasets and the structure to observe if our learning functions can detect the different original parameters and their changes or not.

First we splitted each dataset to 10 parts and we started the incremental learning by feeding to it the 10 parts of each dataset corresponding to each Bayesian Network. The 3 created bayesian networks have the same structure as the one in fig. 3.3.

To assess the performace of the incremental learning on Non-Stationary like-system, we observe the learning process of the variable E and compare it to its original probability table.

The original probability tables for the variable E in the 3 created BNs are in fig. 3.3 and fig. 3.4 and 3.5.

As we can see in fig. 3.6, $\Pr(E)=\text{True}$ was 0.4 approximately at the beginning of the first iteration and converged to a value near 0.1 at the end the 10th iteration (like the original probability value in the BN1), and then it was able to self-adapt to reach the value of 0.9 at the end of the 20th, and then re-adapted itself to reach a value near 0.4 at the end of the 30th iteration.

The above experiment was done with a forgetting factor = 0.7, which

B C		E	
		T	F
F	F	0.1	0.9
F	T	0.2	0.8
T	F	0.01	0.99
T	T	0.7	0.3

TABLE 3.3: Original Probability table of variable E in the first Bayesian Network of Dummy Usecase given B and C

B C		E	
		T	F
F	F	0.9	0.1
F	T	0.8	0.2
T	F	0.99	0.01
T	T	0.7	0.3

TABLE 3.4: Original Probability table of variable E in the second Bayesian Network of Dummy Usecase given B and C

B C		E	
		T	F
F	F	0.4	0.6
F	T	0.5	0.5
T	F	0.4	0.6
T	T	0.3	0.7

TABLE 3.5: Original Probability table of variable E in the third Bayesian Network of Dummy Usecase given B and C

means it takes several 70% of the old learnt parameters and 30% of the new parameters. Of course, if we made this forgetting factor as low as possible, like 0.2, we would expect the learning process act as non-incremental and converges very quickly to the correct values as shown in fig. 3.7.

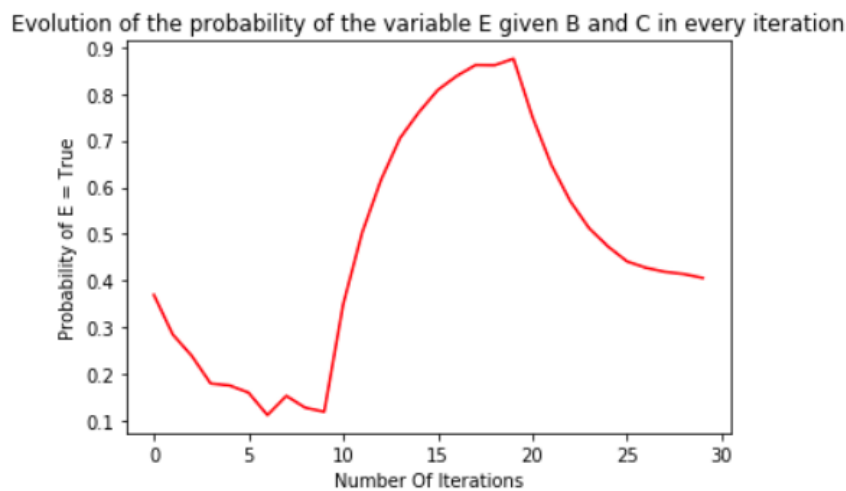


FIGURE 3.6: The evolution of the learnt probability of variable $E = T$ given $B = F, C = F$

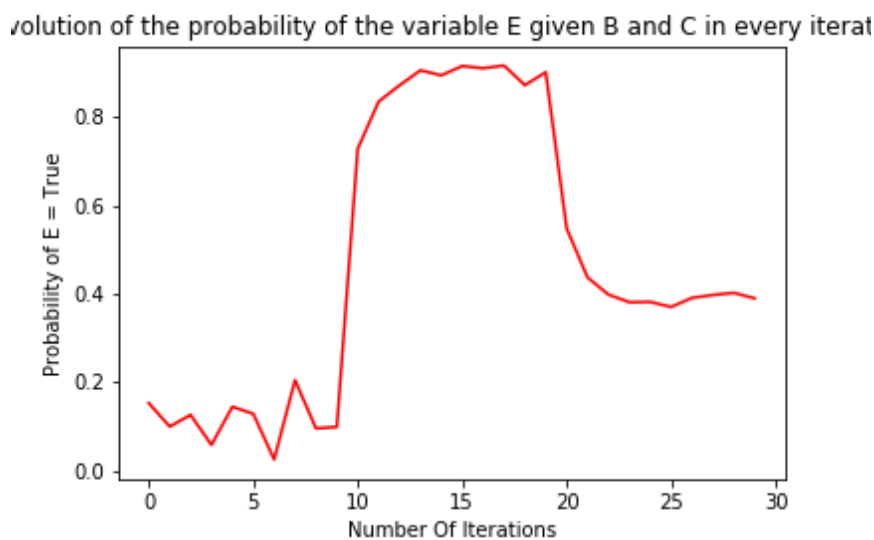


FIGURE 3.7: The evolution of the learnt probability of variable $E = T$ given $B = F, C = F$ when using a forgetting factor - 0.2

3.6 Conclusion

In this chapter, I focused on the Bayesian Network Parameter Learning. I implemented in Pilgrim:

1. Created usecases (Dummy usecase and EyeColorGene usecase) to test the Parameter Learning.
2. Added the incremental learning functionality to the learning functions in PILGRIM.
3. Tested the incremental learning on complete data

4. Tested the incremental learning on incomplete data (like the Noisy Max)
5. Tested the incremental learning with different forgetting factors.
6. Tested the incremental learning with different data that came from different models to see whether it converges at the end. (Non-stationary systems)

Chapter 4

Non-Incremental Structure Learning

4.1 Introduction

Bayesian Network Structure Learning aims at learning the structure of a network given a dataset. It can be learned from experts but in many applications, experts are not available and it should be learned from data.

Learning a Bayesian Network structure is a complex task. Given n variables, the number of possible structures $g(n)$ can be equal to [13]:

$$g(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ \sum_{i=1}^n (-1)^{i+1} \binom{n}{i} 2^{i(n-1)} g(n-1) & \text{if } n > 1 \end{cases}$$

4.1.1 Score based methods

One of the most used Structure learning methods (especially in PILGRIM library) is the score-and-search based methods.

These methods search through different possible structures and DAGs for the ones that maximize a scoring function.

Example of these methods:

1. Maximum Weight Spanning Tree (MWST)[5]
2. Greedy Equivalence (GES)[4]
3. Greedy Search (GS)[4]
4. K2[6]

A *scoring function*: is a function that evaluates a structure or a DAG and assigns value to it. This value represents how well the structure fits the data. So, assuming a DAG g and data D , $P(G|D)$ is the likelihood, so the score function is:

$$Score(G, D) = P(G|D)$$

Many scoring functions can be used in PILGRIM, for example:

1. Bayesian Information Criteria (BIC)[14]

2. Akaike's Information Criteria (AIC)[1]
3. Bayesian Dirichlet equivalent (BDe)[6]
4. Minimum Description Length (MDL)[12]

4.1.2 Constraint Based methods

These methods identify conditional independencies with statistical tests. One of the most known constraint based algorithms is the PC algorithm [15]. In these methods, statistical tests are performed on which depends the removal or the keep of edges. Examples of the statistical tests used are : Mutulal Information (MI), X^2 .

4.1.3 Hybrid methods

In these methods, both score based methods and constraint based methods are combined together for bayesian network learning. So, we can use the output of the constraint based methods as an input to the score based methods. One of the hybrid methods are the MMHC algorithm (MaxMin Hill Climbing)[16]. This algorithm was proposed to solve the high dimensionality problem.

In this algorithm:

1. We build an undirected graph called graphCPC that shows which nodes are directly linked, without knowing which is the parent and which is the child (From here came the noun "CPC" = Candidate Parent Children).
For this step the MMPC algorithm is called for each node T that returns to us the $CPC(T)$ = The set of the candidate parents or children for this node
2. Then, a greedy search (GS) algorithm is called, that searches for the BN structure, knowing the GraphCPC. This GraphCPC will represent constraints to the searching process.

We are interested here in the first step, the MMPC algorithm.

MMPC and MMPCbar

Assume we have two nodes, X1 and X2. If the graph $CPC(X1)$ contains node X2, so the graph $CPC(X2)$ should have node X1.

So our MMPC algorithm calls MMPCbar algorithm to estimate the CPC graphs, then MMPC algorithm corrects the graphs to have this aspect of symmetry.

```

CPC(T) = MMPCbar(T);
for each node X in CPC(T) do
    if T is not in MMPCbar(X) then
        Remove node X from CPC(T);
    end
end

```

Algorithm 2: MMPC algorithm

So we can say that MMPCbar is our "real" algorithm while MMPC is just correcting the symmetry.

Follows is the algorithm of MMPCbar[16]:

```

Data: Target Variable T and Data D
Result: CPC(T)
CPC =  $\emptyset$ ;
repeat
     $\langle F, \text{assocF} \rangle = \max_{x \in X \setminus T} (\min_{S \subseteq \text{CPC}} \text{Assoc}(X; T | S));$ 
    if  $\text{assocF} \neq 0$  then
        Add(CPC, F);
    end
until UNTIL CPC has not changed;
for all  $X \in \text{CPC}$  do
    if  $\exists S \subseteq \text{CPC}, \text{s.t. } \text{Assoc}(X; T | S) = 0$  then
         $\text{CPC} = \text{CPC} \setminus X;$ 
    end
end
return CPC

```

Algorithm 3: MMPC algorithm

The first loop is a forward a phase while the second loop is a backward phase. In the forward phase, we try to build the CPC graph and add nodes to it. The backward phase we remove the False Positives, since that in the forward phase we calculate the assoc given the subset already discovered and at each iteration of the loop the subset is not always complete. Assoc() is a dependency measure function that measures the dependency between node X and T. So $\text{Assoc}(X, T) = 0$ if X and T are independent nodes. In our implementation this dependency measure was represented by the MutualInformation measure. $\min_{S \subseteq \text{CPC}} \text{Assoc}(X; T | S)$ finds the DIRECT links between X and T given the CPC already discovered.

Only one problem, that when finding the DIRECT links between nodes given the CPC, maybe that CPC is filled with false nodes. So here comes the backward phase that will test all the nodes put in the CPC and remove those who are false.

Please Note that the forward phase can be seen as a greedy search process as shown in fig. 4.1

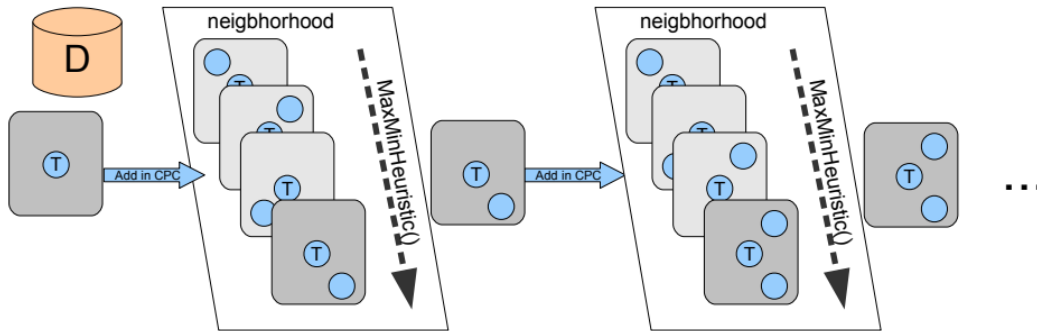


FIGURE 4.1: The forward phase that can be viewed like a greedy search

Source: *Incremental Bayesian network structure learning from data streams* by Amanullah Yasin [17]

4.2 Contribution

Concerning the tasks related to the Bayesian Network non-incremental Structure Learning in PILGRIM, several usecases were built to show the approach used to implement and learn a BN structure in PILGRIM.

The approach was as follows:

1. Get the original DAG of a usecase by loading it or creating it manually
2. Generate Data from the original DAG
3. Specify Scoring metric (BIC, AIC, BDeu, QNML, MDL)
4. Specify whether an init BN will be used or not. If initBN will be used it can be:
 - (a) A random graph generated using the PMMixed algorithm[9]
 - (b) A smart Bayesian network learnt using another structure learning algorithm in order to make the learning process faster
5. Specify whether an Graph CPC will be used or not to put constraints on the structure learnt.
6. Choose the structure learning algorithm and start it.
7. Evaluate the learnt network by making a comparison using TP, FP, TN, FN and SHD

I tested the above approach as well as debugged, tested and fixed the functionality of:

1. converting a DAG into a CPDAG
2. Comparing two CPDAGs in order to compare the original Structure with the learnt one. That was done by:
 - (a) Converting the Original DAG to a CPDAG
 - (b) Converting the learnt DAG to a CPDAG
 - (c) Calculating the SHD between both CPDAGs
 - (d) Calculating TP (True Positive), FP (False Positive), TN (True Negative), FN (False Negative) of the DAGs

4.3 Experiments and Results

In our experiments we tested on several examples and usecases. The one demonstrated here is the Asia example, presented by Lauritzen & Spiegelhalter(1988)[10], which consists of several binary variables that describe a simple diagnostic problem for a couple of diseases. In this section I would like to show you some of the results of the structure learning process on this usecase example.

4.3.1 Original Structure of Asia Usecase

DAG

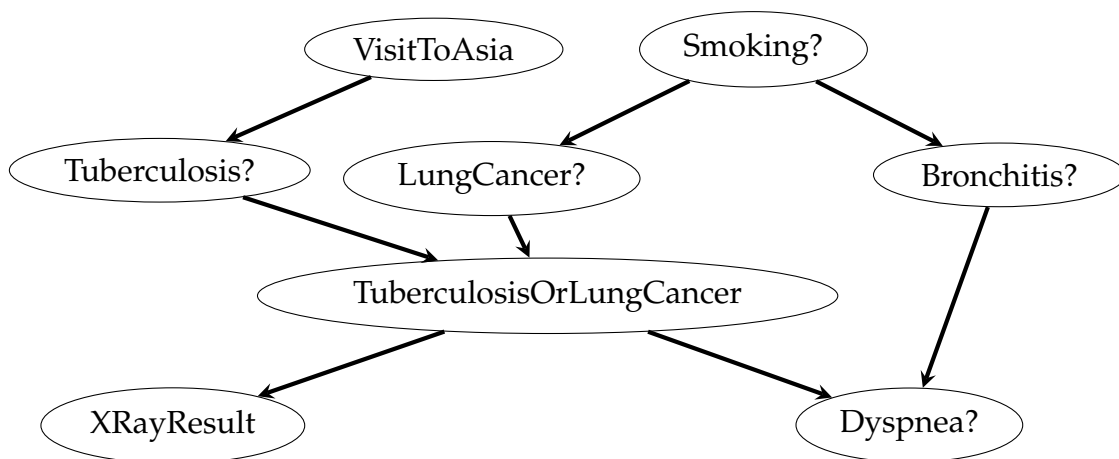


FIGURE 4.2: Original Structure of Asia Usecase

CPDAG

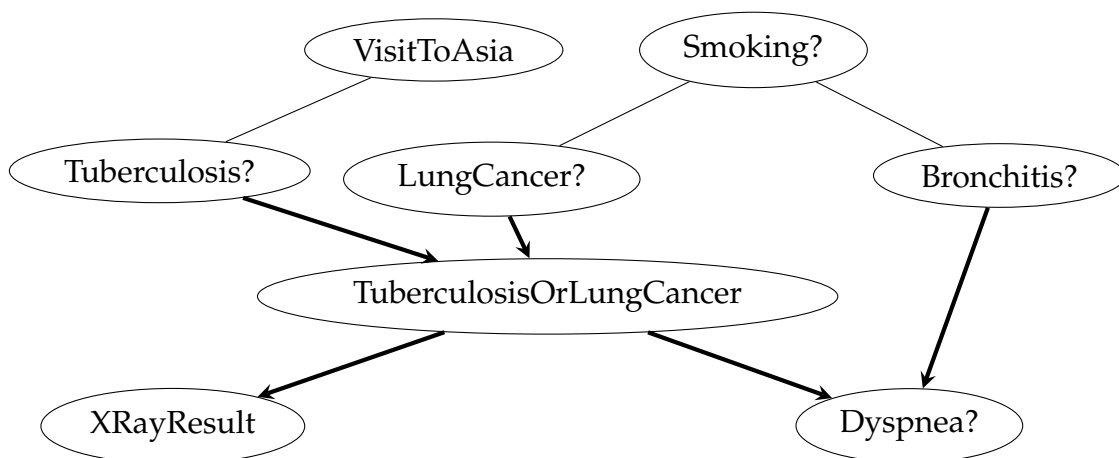


FIGURE 4.3: CPDAG of the Original Structure of Asia Usecase

To execute the learning algorithms, I generated data from the original structure, the data size of 1000 records.

4.3.2 Asia Structure Learning using MWST algorithm

DAG

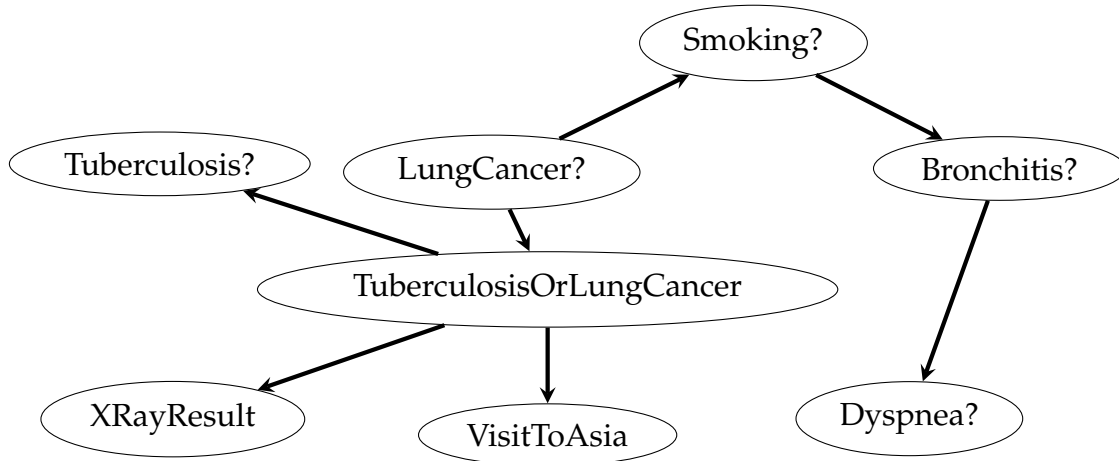


FIGURE 4.4: Asia Learnt Structure using MWST algorithm

By evaluating the learnt structure in 4.4 using PILGRIM and comparing it with the original one, we find that:

1. TP = 6. This is because of:
 - (a) Tuberculosis – TuberculosisOrLungCancer
 - (b) TuberculosisOrLungCancer – XRayResult
 - (c) TuberculosisOrLungCancer – LungCancer
 - (d) LungCancer – Smoking
 - (e) Smoking – Bronchitis
 - (f) Bronchitis – Dyspnea
2. FP = 1. This is because of:
 - (a) TuberculosisOrLungCancer – VisitToAsia
3. FN = 1. This is because of:
 - (a) TuberculosisOrLungCancer – Dyspnea
4. TN = 19

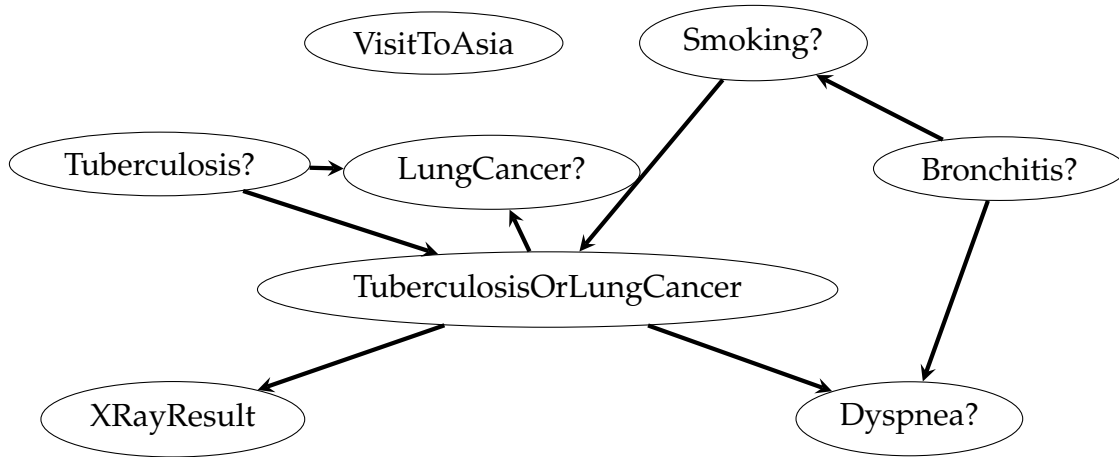


FIGURE 4.5: Learned Structure of Asia Usecase using GS without init

4.3.3 Learnt Structure of Asia Usecase using GS algorithm

DAG

By evaluating the learnt structure in 4.5:

1. TP = 6 because of:
 - (a) Tuberculosis -> TuberculosisOrLungCancer
 - (b) TuberculosisOrLungCancer - XRayResult
 - (c) TuberculosisOrLungCancer - Dyspnea
 - (d) TuberculosisOrLungCancer - LungCancer
 - (e) Smoking - Bronchitis
 - (f) Bronchitis - Dyspnea
2. FP = 2 because of:
 - (a) Smoking - TuberculosisOrLungCancer
 - (b) Tuberculosis - LungCancer
3. FN = 2 because of:
 - (a) Smoking - LungCancer
 - (b) VisitToAsia - Tuberculosis
4. TN = 18

CPDAG

By calculating the SHD by comparing the the original CPDAG with the CPDAG in 4.6 we found that the SHD = 5, this because of:

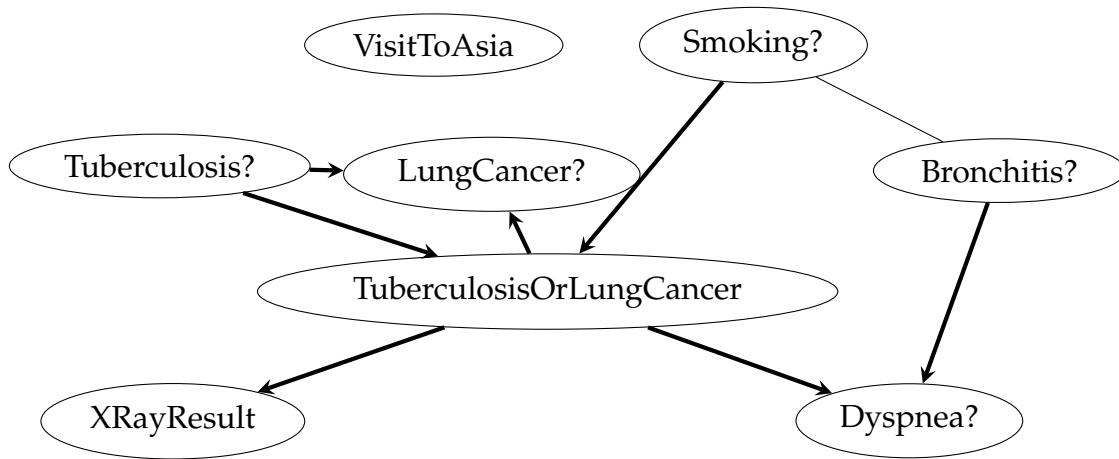


FIGURE 4.6: CPDAG of the Learnt Structure of Asia Usecase using GS without init nor Graph CPC

1. VisitToAsia – Tuberculosis
2. LungCancer – Smoking
3. Tuberculosis – LungCancer
4. Smoking – TuberculosisOrLungCancer
5. LungCancer – TuberculosisOrLungCancer

4.3.4 Asia Structure Learning using Greedy Search algorithm + Random Init

The DAG of the Learnt Structure

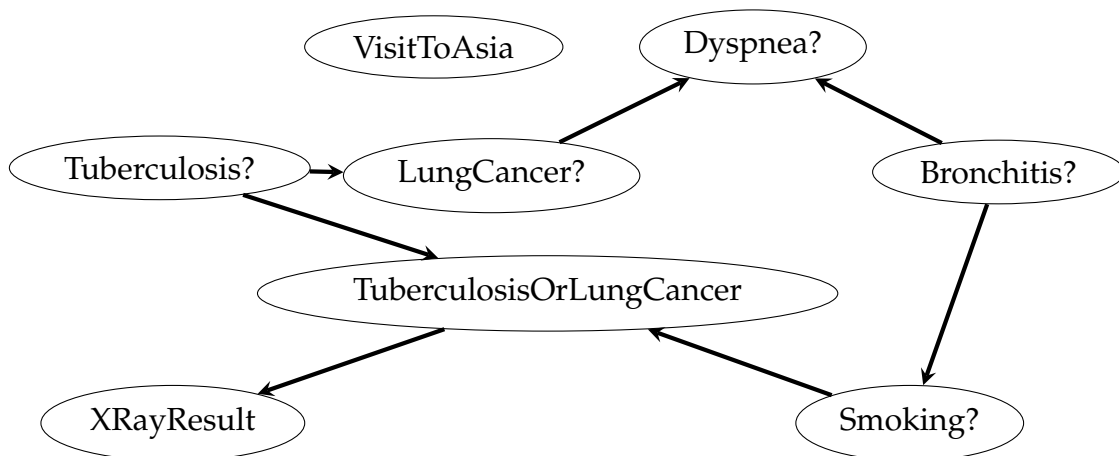


FIGURE 4.7: Asia Learnt Structure using GS algorithm + Random Graph Init by PMMixed

By evaluating the learnt structure in 4.7 and comparing it to the structure of the original DAG using PILGRIM, we find that:

1. TP = 5. This is because of:
 - (a) Tuberculosis – TuberculosisOrLungCancer
 - (b) LungCancer – TuberculosisOrLungCancer
 - (c) TuberculosisOrLungCancer – XRayResult
 - (d) Smoking – Bronchitis
 - (e) Bronchitis – Dyspnea
2. FP = 3. This is because of:
 - (a) Tuberculosis – LungCancer
 - (b) LungCancer – Dyspnea
 - (c) Smoking – TuberculosisOrLungCancer
3. TN = 17.
4. FN = 3

CPDAG of the learnt structure

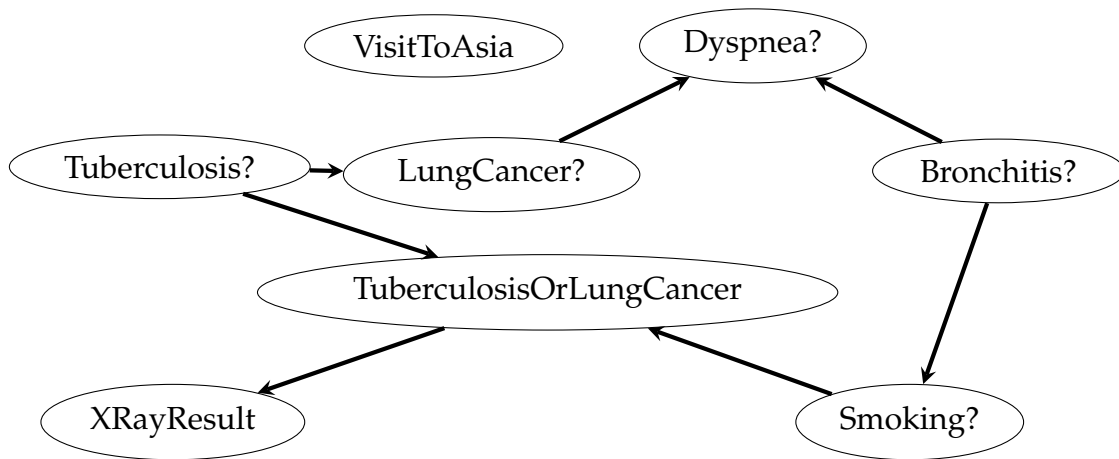


FIGURE 4.8: CPDAG Asia Learnt Structure using GS algorithm + Random Graph Init by PMMixed

Calculating the SHD yields to SHD = 7. This is because of:

1. VisitToAsia – Tuberculosis
2. Tuberculosis – LungCancer
3. Smoking – TuberculosisOrLungCancer
4. TuberculosisOrLungCancer – LungCancer

5. Smoking – LungCancer
6. LungCancer – Dyspnea
7. TuberculosisOrLungCancer – Dyspnea

The Init Random Network Structure by PMMixed

The initialisation random graph generated by PMMixed algorithm is in fig. 4.7:

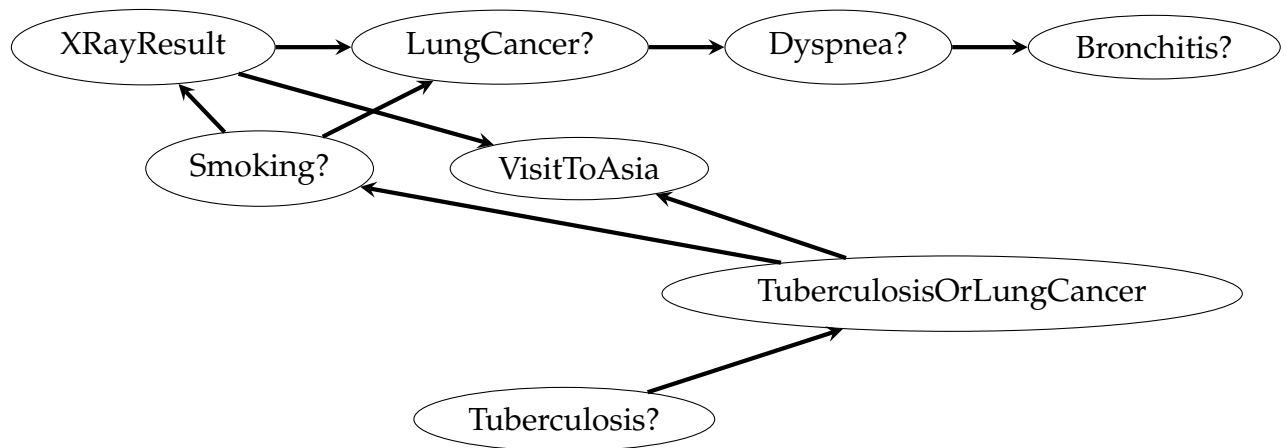


FIGURE 4.9: The random graph generated by PMMixed used as initialisation graph

4.3.5 Asia Structure Learning using Greedy Search algorithm + Smart Init by MWST algorithm

The Learnt Structure DAG

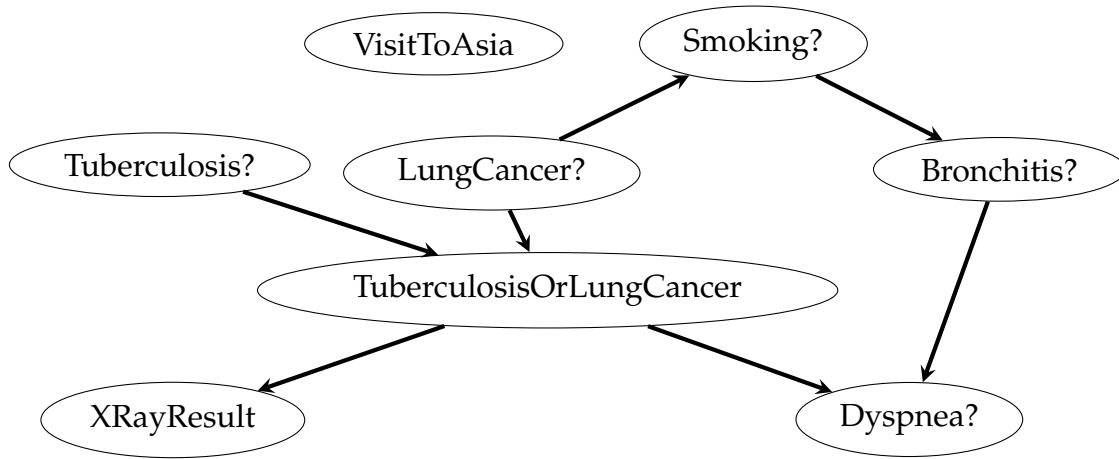


FIGURE 4.10: Asia Learnt Structure using GS algorithm +
Learnt Init Graph learnt using MWST algorithm

By comparing the skeleton of the learnt structure in 4.10 with the skeleton of the original DAG, we can find that:

1. TP = 7. This is because of:
 - (a) Tuberculosis – TuberculosisOrLungCancer
 - (b) LungCancer – TuberculosisOrLungCancer
 - (c) TuberculosisOrLungCancer – XRayResult
 - (d) TuberculosisOrLungCancer – Dyspnea
 - (e) Smoking – LungCancer
 - (f) Smoking – Bronchitis
 - (g) Bronchitis – Dyspnea
2. FP = 0
3. FN = 1 this is because of a missing edge in the learnt structure which is:
 - Tuberculosis – VisitToAsia
4. TN = 20

The CPDAG of the Learnt Structure

The SHD is equal to 1 and this is due to: Tuberculosis – VisitToAsia

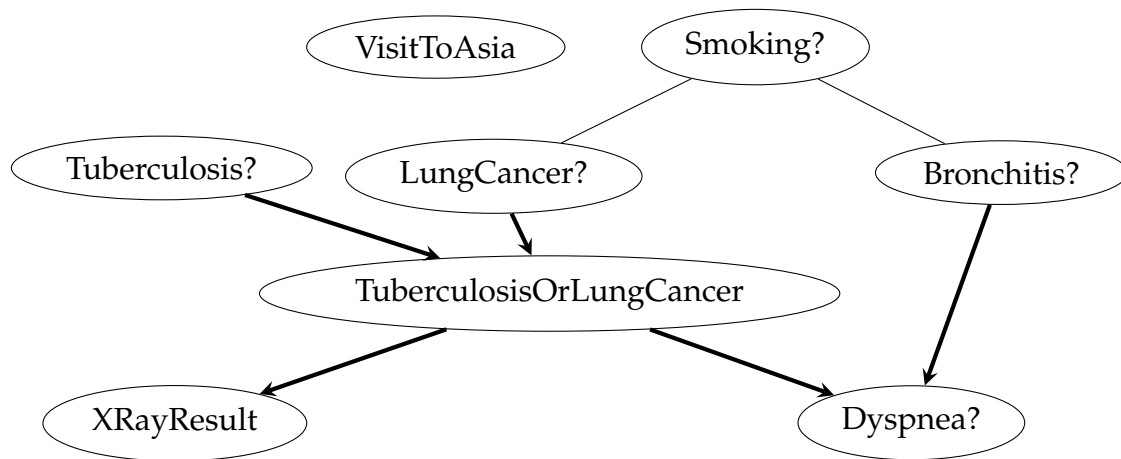


FIGURE 4.11: CPDAG of the Asia Learnt Structure using GS algorithm with a mwst learnt structure as initial BN

The Init Structure generated by MWST

The initialisation graph was obtained by learning the structure of the same usecase and the same data using a different algorithm (MWST). This graph is described in section [4.3.2](#)

4.4 Conclusion

In this chapter, I represented some of the Structure Learning tasks that I accomplished. It was all related to non-incremental structural learning.

I contributed to Pilgrim by:

1. Created usecases to learn the structure of a BN using Greedy Search algorithm (GS) or MWST algorithm.
2. Debugged and maintained the BN Comparison metrics calculations (SHD, TP, TN, FP, FN)
3. Debugged and maintained the chickering algorithm
4. Integrated the PMMixed algorithm that generates random graphs giving some constraints.
5. Modifying some of the functions in PILGRIM so that a BN can be learned from an initial BN. This initial BN can be learnt from another learning algorithm like MWST algorithm or generated randomly from the PM-Mixed algorithm

Chapter 5

Incremental Structure Learning

The work we have done in the incremental structural learning part was based on the work of the PhD thesis of Amanullah Yasin [17]. His work was also done under the supervision of my master internship supervisor, Prof. Philippe LERAY.

5.1 Roure's approach (TOCO and RSS)[17]

Roure proposed an incremental bayesian network approach for data streams. The work of Amanullah Yasin along with Prof. Philippe Leray in this area was to prove that the forward phase of the mmpcbar algorithm can be viewed as a greedy search process. Then, apply Roure's approach on the forward phase (TOCO+RSS with sufficient statistics).

5.1.1 TOCO Heuristic

In this section, an overview will be given on TOCO Heuristic that will help explain the Incremental MMPC algorithm later.

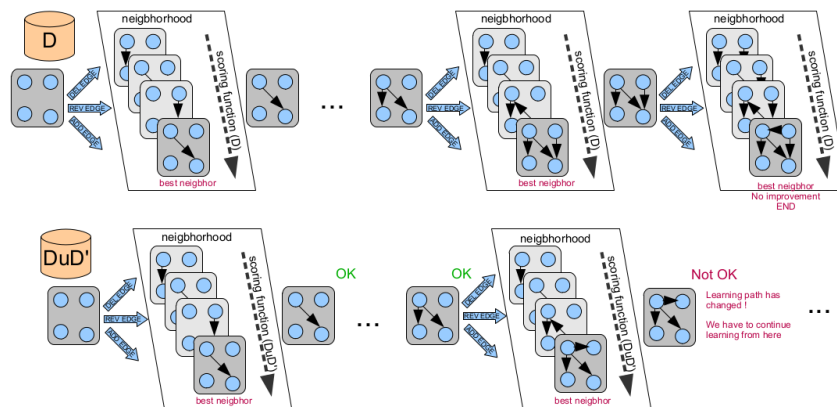


FIGURE 5.1: Roure's TOCO Heuristic represented by Amanullah Yasin in his thesis

Source: *Incremental Bayesian network structure learning from data streams* by Amanullah Yasin [17]

In the first part in fig. 5.2, given a first Dataset D we have an empty graph, we search for the best node that maximizes a score function and we continue

like that as an optimization for the scoring function. In the second part in the same figure, we got a first model in the first part, and after being introduced to a second Dataset D' , we would like to verify that the set of operations done on D are still interesting and can be used to learn the network that corresponds to the data. Once we discovered that the operations are no more the same we begin the structure learning using greedy search from the union of the 2 datasets $D \cup D'$.

5.1.2 RSS Heuristic

In this section, an overview will be given on RSS Heuristic that will help explain the Incremental MMPC algorithm later.

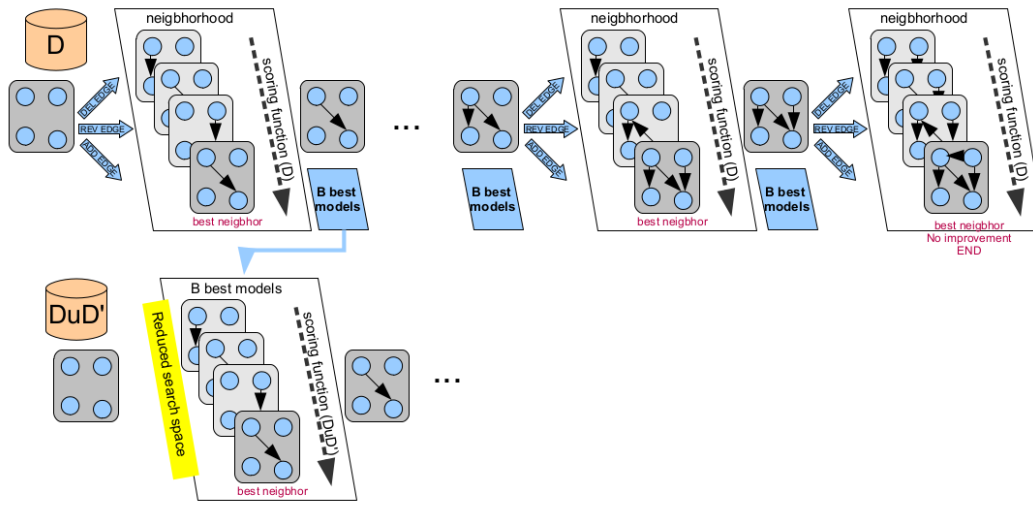


FIGURE 5.2: Roure's RSS Heuristic represented by Amanullah Yasin in his thesis

Source: *Incremental Bayesian network structure learning from data streams* by Amanullah Yasin [17]

The problem with using only TOCO Heuristic that in order to make sure that the set of operations done on D are still interesting on for $D \cup D'$, we have to redo all the learning process and the scores calculations from the beginning. That's why the RSS heuristic was proposed.

At each step of the search path, it stores the top n RSS models in a set B having the score closer to the best one. (The n RSS is a parameter that limits the search space to avoid exploring the parts where low quality models were found before.) So, in TOCO, instead of verifying that a given model is the best model among all models, we verify that it is the best model among the "best models" in the set B .

5.2 Incremental MMPC

In the fig. 5.3, the work done by Amanullah Yasin for the MMPC 4.1.3 and the integration of the TOCO+RSS.

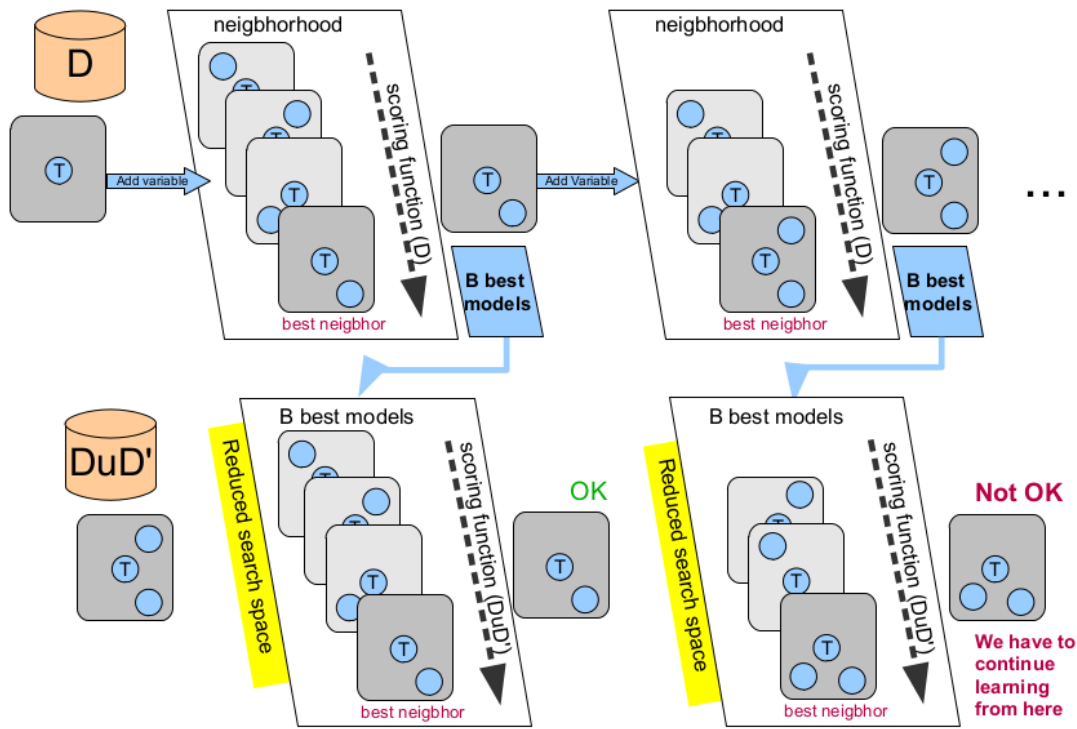


FIGURE 5.3: Outling of IMMPC

Source: Incremental Bayesian network structure learning from data streams by Amanullah Yasin [17]

So instead of searching for the best CPC graphs among all the nodes, the search will be reduced to be among only the B models.

The algorithm works as the following:

The model = a set of CPCs + a target variable.

Initially, the model M_0 contains only the target variable and its CPC is empty. Then, it adds in the $CPC(T)$ the variables who have strong dependencies with the target variable using the MaxMinHeuristic described in the algorithm of MMPC.

The algorithm then adds the variables that has the best dependency with the target variable given the discovered neighbours. This process is repeated until no other variables can be added.

At each step, we keep the K best variables having dependency scores close to the variable added in the model. This K best variables constitute our B best models. When having new data, we consider our initial M_0 to be the previously learned search path and checks whether our previously added variables in the list of $CPC(T)$ are valid for the new data $D \cup D'$

5.3 Calculating Sufficient Statistics[17]

When executing the IMMPC, in the second phase presented in fig. 5.3, not D' only that is used but it's $D \cup D'$. It can be a bit complex to store all data for the next phase, that's why sufficient statistics are calculated (Sufficient

statistics are the number of occurrences of the variables or a set of variables). This step resembles the step of the parameters incremental learning.

So for example, assume we have $N(X1, X2)$ a contingency table for variables $X1$ and $X2$ for the Dataset D and $N''(X1, X2)$ the contingency table for the same variables but over the Dataset $D \cup D'$ and $N'(X1, X2)$ the contingency table for the same variable over the Dataset D' . If we are in a non-incremental case, normally we should have:

$$N''(X1, X2) = N(X1, X2) + N'(X1, X2)$$

To be able to do these calculations when receiving a new dataset D' , we should stock the contingency tables calculated on D in some sort of a cache. If we don't have the contingency tables in the cache, we can still be able to calculate it or at least have some approximations of it using the learnt model on the Dataset D . This is done by querying the distribution of the previous Model. So from the previous learnt model we get: $P(X1, X2) * N_{old}$. (N_{old} is the number of the records of the previous dataset)

In an incremental case, these calculations will differ a little since we will add a forgetting factor for the calculations of the sufficient statistics but also for the calculations of N .

$$N''(X1, X2) = N(X1, X2) * ForgettingFactor + N'(X1, X2)$$

As shown in the fig. 5.4, we can see that how the frequency counter is used as a summary of the data itself, in order to avoid storing all the data which can be inefficient.

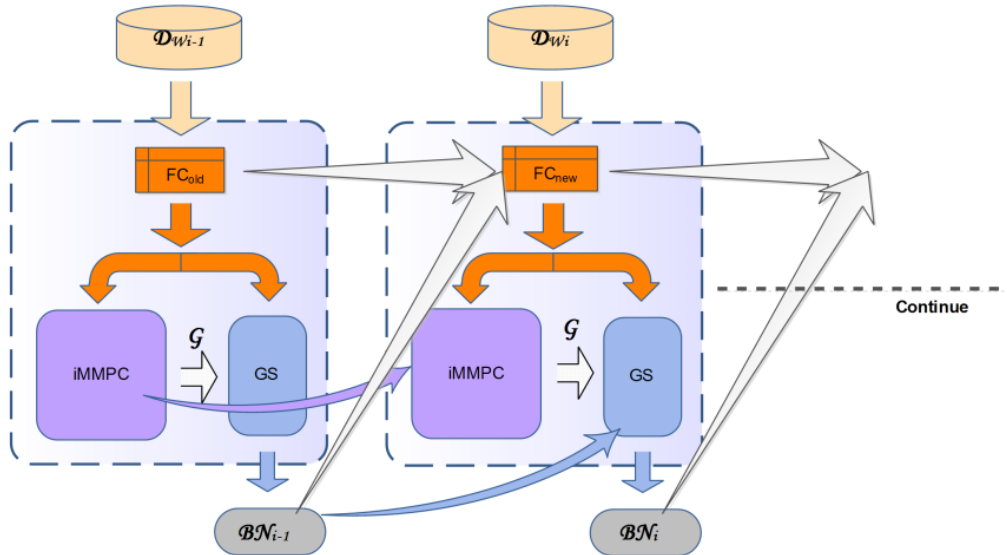


FIGURE 5.4: Showing the usage of the FrequencyCounter
Source: Incremental Bayesian network structure learning from data streams by Amanullah Yasin [17]

So in the figure 5.4, instead of storing the $D_{wi-1} \cup D_{wi}$ for the next iteration, we feed to the algorithm only D_{wi} and get the stored sufficient statistics that represent the old data as well as the previous Bayesian network. The new sufficient statistics (calculated from the current data and the previous sufficient statistics and the previous network) are fed to the IMMPC algorithm as well as to the GS algorithm.

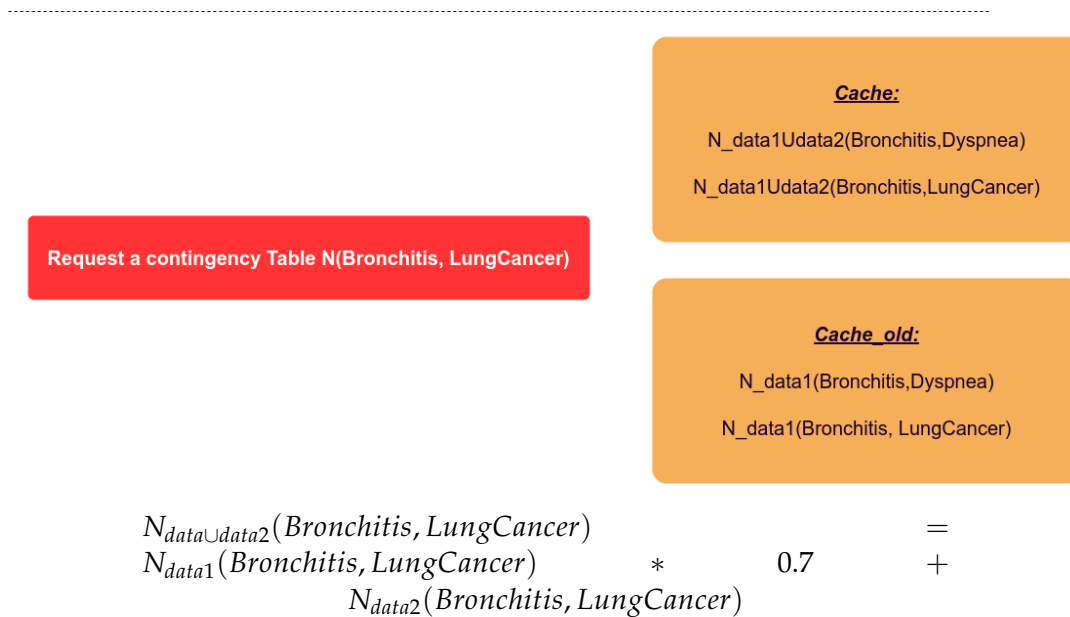
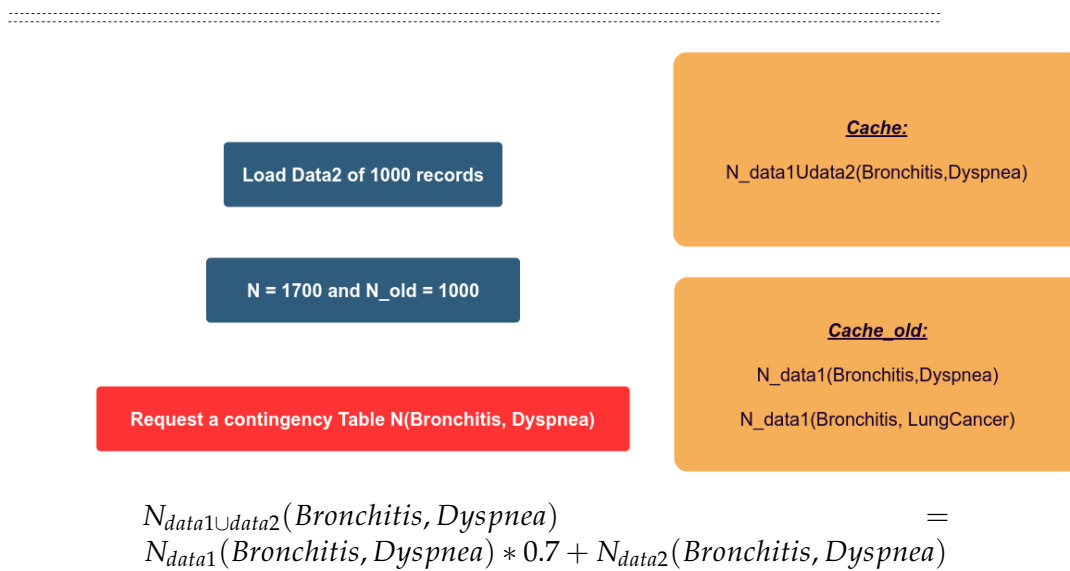
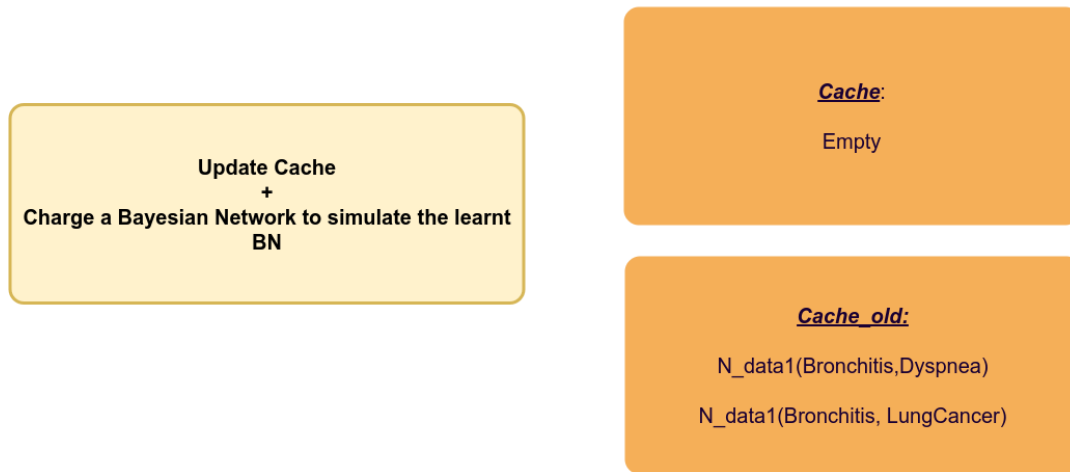
In the figure, the FC_{old} and FC_{new} represent a class called Frequency Counter (implemented in PILGRIM) that stocks the sufficient statistics.

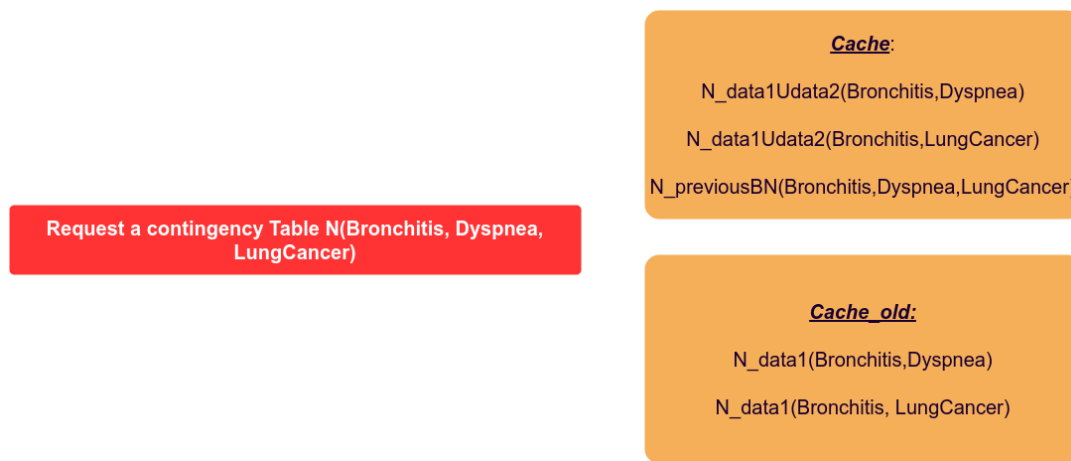
5.4 Contribution

5.4.1 Sufficient Statistics

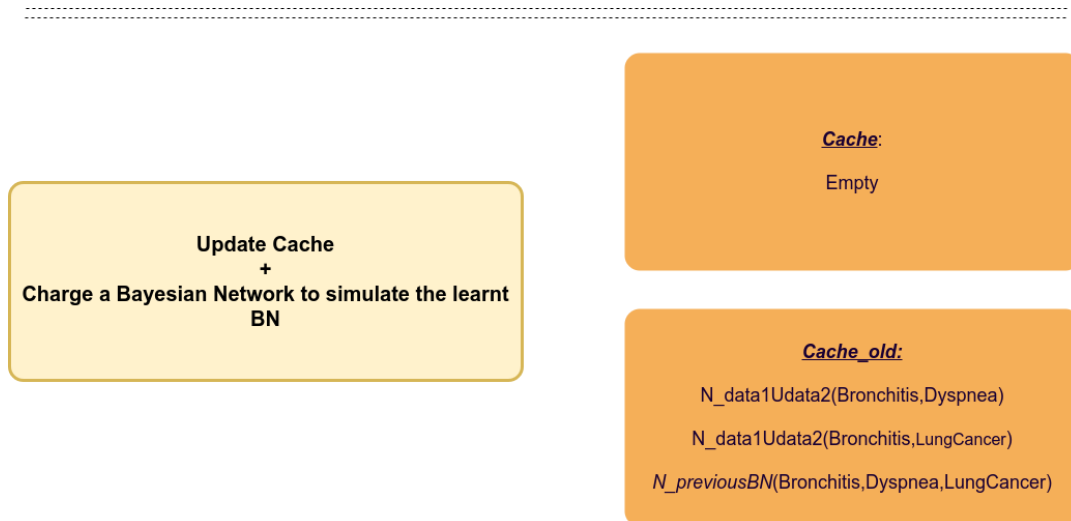
In my work that concerns this part, I re-implemented and integrated 2 important classes into Pilgrim, frequencyCounter and SlidingWindow, these classes serve to calculate the sufficient statistics in order to be able to use them later in the IMMPC algorithm. I tested them with the Asia dataset described in 4.2. The results of the log were as expected and the steps consisted of:







$$N_{\text{previousBN}}(\text{Bronchitis}, \text{Dyspnea}, \text{LungCancer}) (= = N_{\text{data1} \cup \text{data2}}(\text{Bronchitis}, \text{Dyspnea}, \text{LungCancer}))$$

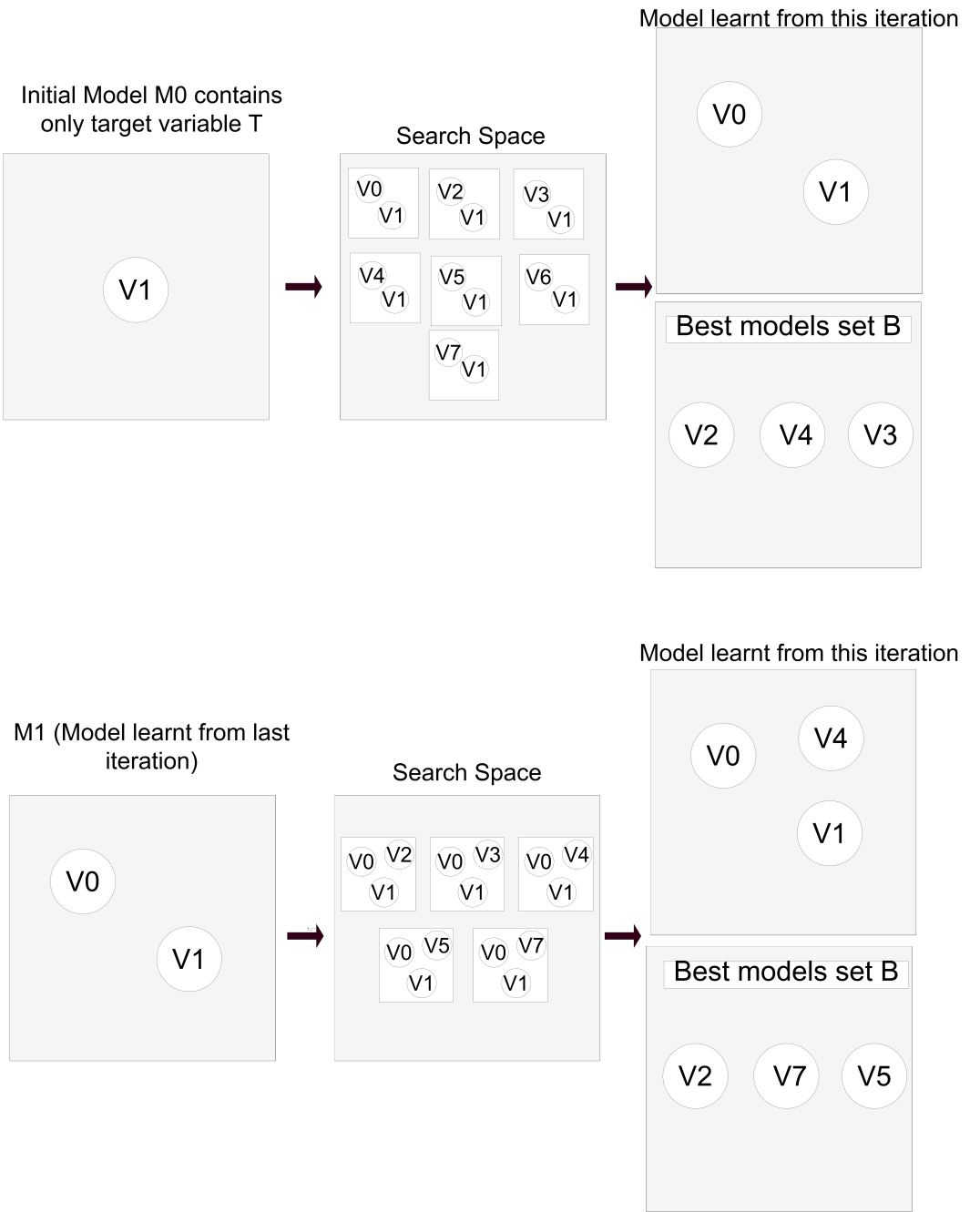


5.4.2 IMMPC and IMMPCbar

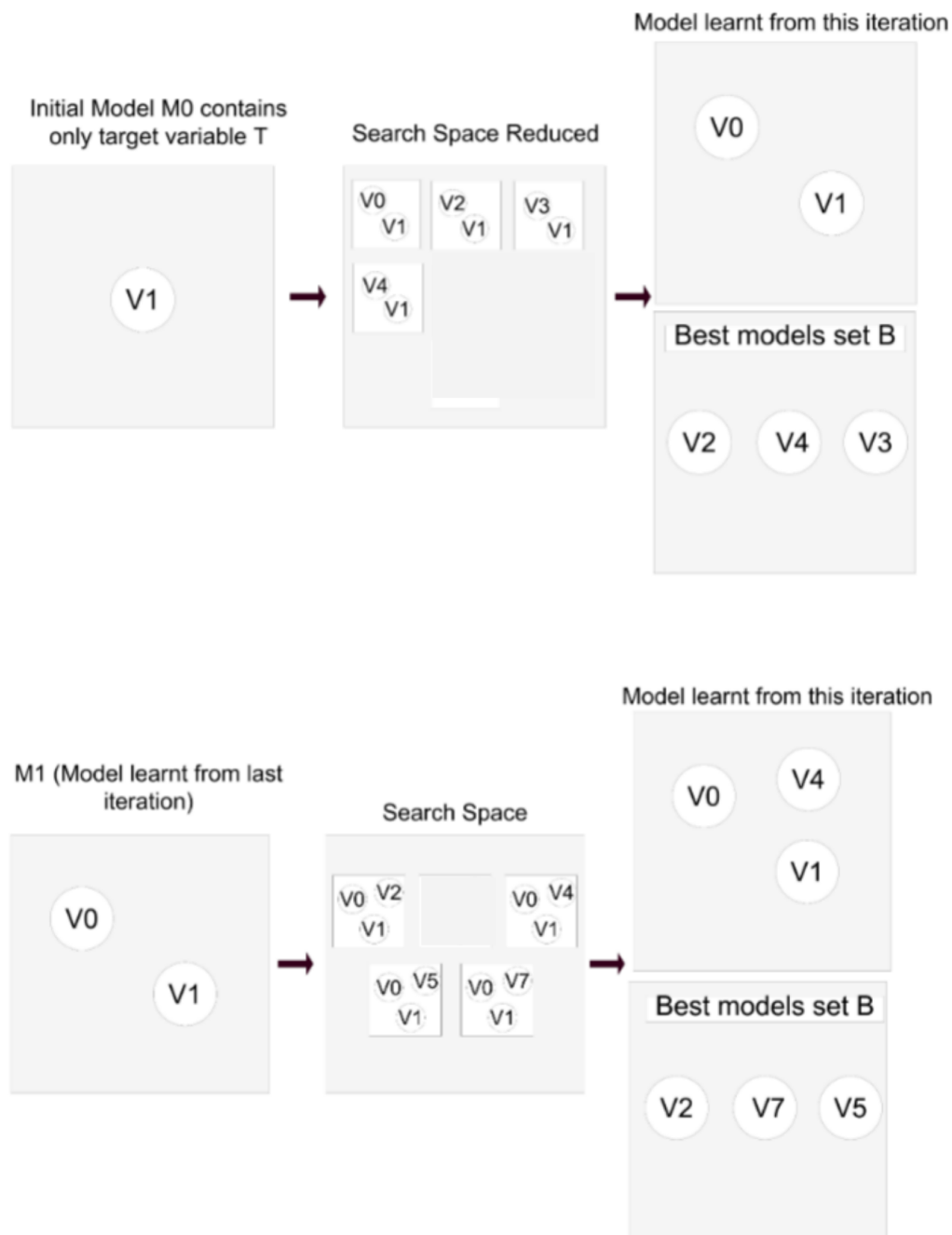
In my work, I finally integrated then debugged and tested the IMMPC and IMMPCbar to identify problems and make sure of their functionalities and their working with the MMHC algorithm and with the other Pilgrim/General library classes.

Example:

IMMPC on dataset D



IMMPC on dataset DuD'



The previous example is an example that was done on the Asia dataset whose graph represented in 4.2.

1. v0 -> Bronchitis
2. v1 -> Dyspnea
3. v3 -> Smoking

4. v4 -> TborCa
5. v5 -> Tuberculosis
6. v6 -> VisitToAsia
7. v7 -> XRay

The data fed to the algorithm is a data of 1000 records for each ($D = 1000$ records and $D' = 1000$ records).

The first part of the figures is the simulation of the algorithm on D , while the second part of the figures is the simulation of the algorithm on $D \cup D'$. First case, we will search the whole search space (all the variables), while in the second case we will investigate a reduced search space (the set B) thanks to RSS.

In the first case:

In the first iteration, we begin by an initial Model M_0 that contains only the target variable which is $V1$ and an empty set of CPC. Dependency measures calculations ($\text{minAssoc}()$) are done on each of the other variables except the target variable. This yields to discovering that $V0$ is a neighbour of $V1$ - the maximum value of $\text{minAssoc}()$. Also, we store the best variables that have the association value close to the association value($V0, V1$).

In the second iteration, we begin by the learnt model of the previous iteration M_1 that contains the target variable which is $V1$ and the CPC graph nodes discovered which is $V0$. We proceed by the calculations of the $\text{minAssoc}()$ given the already discovered node to find out that the node $V4$ is the one with the biggest value of $\text{minAssoc}()$.

Please note : Here we calculate the $MI(V1, V2)$ and $MI(V1, V2|V0)$, and the smallest value will become the $\text{minAssoc}()$.

After finishing the iteration, all the rest of the association values are very weak such that they become not interesting.

In the second case: It starts from the initial model that contains only the target variable which is $V1$ and an empty set of CPC.

Search space becomes reduced to the B best models (Set B). The learning path and the search path are verified. If the best neighbour is not the same, we re-begin the learning process from that point.

5.5 Conclusion

In this chapter, I gave an overview over Roure's approach (TOCO + RSS heuristics) and how Amanullah Yasin used them to make the MMPC algorithm incremental and create the IMMPC algorithm. I also showed my contribution by integrating the IMMPC classes into PILGRIM library and

debugging and testing them as well as re-implementing and debugging and finally testing the frequency counter classes and the slidingWindow classes that have a role to calculate the sufficient statistics.

Chapter 6

Thesis Conclusion

In this internship, I had the opportunity to work on different algorithms from both theoretical and implementation perspectives and to contribute to PILGRIM library. In this report, I described most of the tasks and contribution done to PILGRIM library and the algorithms implemented, debugged or tested or integrated. These contributions were:

1. Bayesian network comparison metrics (SHD, TP, FP, FN, TN)
2. Chickering Algorithm
3. PMMixed algorithm
4. Usecases to test the non-incremental learning functionality for complete data
5. Usecases to test the non-incremental learning functionality for incomplete data
6. Implementing the incrementality aspect in the learning functions in PILGRIM
7. Tested the above functionality on Complete Data
8. Tested the above functionality on Incomplete Data
9. Tested the incrementality with different forgetting factors and different number of iterations.
10. Tested the incrementality for NON-Stationary Systems
11. Created Usecases to show the structure learning functionalities in PILGRIM using different variations:
 - Using a random initial Bayesian network with some constraints generated by PMMixed algorithm
 - using an initial Bayesian Network learnt using MWST algorithm
 - using a graphCPC that is considered as a constrained network for the learnt BN
12. MMPC and MMPCbar algorithms for graphCPC generation

13. FrequencyCounter and SlidingWindow classes that handles the sufficient statistics and the caches as described.
14. Integrating the IMMPC to be used along with the IMMHC algorithm for incremental Structural Learning.

During these tasks, I had to modify and re-implement many things in the different classes of PILGRIM library.

In this internship, I was able to discover and to learn different algorithms concerning the BN learning or even concerning different domains around the learning process.

Also, my skills in C++ and Object Oriented programming developed significantly.

I was glad that I was able to work on a such number of algorithms and to go deeper to know how libraries are actually implemented.

Bibliography

- [1] Hirotugu Akaike. “Statistical predictor identification”. In: *Annals of the Institute of Statistical Mathematics* 22.1 (Dec. 1970), pp. 203–217. ISSN: 1572-9052. DOI: [10.1007/BF02506337](https://doi.org/10.1007/BF02506337). URL: <https://doi.org/10.1007/BF02506337>.
- [2] Eric Bauer, Daphne Koller, and Yoram Singer. *Update Rules for Parameter Estimation in Bayesian Networks*. 2013. arXiv: [1302.1519](https://arxiv.org/abs/1302.1519) [cs.LG].
- [3] David Chickering. “Learning Equivalence Classes Of Bayesian Network Structures”. In: *J. Mach. Learn. Res.* 2 (Sept. 2000). DOI: [10.1162/153244302760200696](https://doi.org/10.1162/153244302760200696). URL: <https://doi.org/10.1162/153244302760200696>.
- [4] David Chickering. “Optimal Structure Identification With Greedy Search.” In: *Journal of Machine Learning Research* 3 (Jan. 2002), pp. 507–554. DOI: [10.1162/153244303321897717](https://doi.org/10.1162/153244303321897717). URL: <https://doi.org/10.1162/153244303321897717>.
- [5] C. Chow and C. Liu. “Approximating discrete probability distributions with dependence trees”. In: *Information Theory, IEEE Transactions on* 14.3 (May 1968), pp. 462–467. ISSN: 0018-9448.
- [6] Gregory F. Cooper and Edward Herskovits. “A Bayesian method for the induction of probabilistic networks from data”. In: *Machine Learning* 9.4 (Oct. 1992), pp. 309–347. ISSN: 1573-0565. DOI: [10.1007/BF00994110](https://doi.org/10.1007/BF00994110). URL: <https://doi.org/10.1007/BF00994110>.
- [7] A. P. Dempster, N. M. Laird, and D. B. Rubin. “Maximum likelihood from incomplete data via the EM algorithm”. In: *Journal of the Royal Statistical Society: Series B* 39 (1977), pp. 1–38. URL: <http://web.mit.edu/6.435/www/Dempster77.pdf>.
- [8] Francisco Díez and Marek Druzdzel. “Canonical probabilistic models for knowledge engineering”. In: (May 2007).
- [9] Jaime Ide, Fabio Cozman, and Fabio Ramos. *Generating Random Bayesian Networks with Constraints on Induced Width*. Proceedings of the 16th European Conference on Artificial Intelligence. Jan. 2004, pp. 323–327.
- [10] S. Lauritzen and D. J. Spiegelhalter. “Local computations with probabilities on graphical structures and their application to expert systems (with discussion)”. In: *Journal of the Royal Statistical Society series B* 50 (1988), pp. 157–224.
- [11] Adityanarayanan Radhakrishnan, Liam Solus, and Caroline Uhler. *Counting Markov Equivalence Classes by Number of Immoralities*. 2016. arXiv: [1611.07493](https://arxiv.org/abs/1611.07493) [math.CO].

- [12] Jorma Rissanen. "Modeling by shortest data description." In: *Autom.* 14.5 (1978), pp. 465–471. URL: <http://dblp.uni-trier.de/db/journals/automatica/automatica14.html#Rissanen78>.
- [13] R. W. Robinson. "Counting unlabeled acyclic digraphs". In: *Combinatorial Mathematics V*. Ed. by Charles H. C. Little. Berlin, Heidelberg: Springer Berlin Heidelberg, 1977, pp. 28–43. ISBN: 978-3-540-37020-8.
- [14] G. Schwarz. "Estimating the dimension of a model". In: *The Annals of Statistics* 6 (1978), pp. 461–464.
- [15] P. Spirtes, C. Glymour, and R. Scheines. *Causation, Prediction, and Search*. 2nd. MIT press, 2000.
- [16] Ioannis Tsamardinos et al. "Scaling-Up Bayesian Network Learning to Thousands of Variables Using Local Learning Techniques". In: (Apr. 2003).
- [17] Amanullah Yasin. "Incremental Bayesian network structure learning from data streams". PhD thesis. Oct. 2013.